User Journey Data Flow - FlowCRM

Prepared by: Data Analyst
Date: 17.02.2026
Source: DB Schema · clients_client (120 rows) · Data Quality Report · FlowCRM Scope Baseline

Journey 1: Create New Client

Table: clients_client | **DB Operation:** INSERT

Steps

1. **User Action** —> User clicks 'Add Client'. Form appears with fields: First Name (required), Last Name (required), Email (required), Phone (optional), Comment (optional).
2. **Frontend Validation** —>First Name & Last Name: required, min 2 chars. Email: required, valid format. Phone: optional, 7–15 chars if entered. Comment: max 300 chars. If invalid → inline error shown, request NOT sent.
3. **API Request** —> POST /api/clients/ → Payload: first_name, last_name, email, phone, comment, business_id. Status auto-set to 'New' by backend.
4. **Backend Validation** —> Re-validate all fields. Check email UNIQUE within business_id. Check business_id exists in business_business. Sanitize inputs.
5. **DB Operation** —> INSERT INTO clients_client: id (auto PK), first_name, last_name, email, phone, status='New', comment, deleted=NULL, deleted_by_cascade=FALSE, created_at=NOW(), updated_at=NOW(), business_id (FK).
6. **Result** Created → client card appears in 'New' Kanban column within ≤ 3 sec.

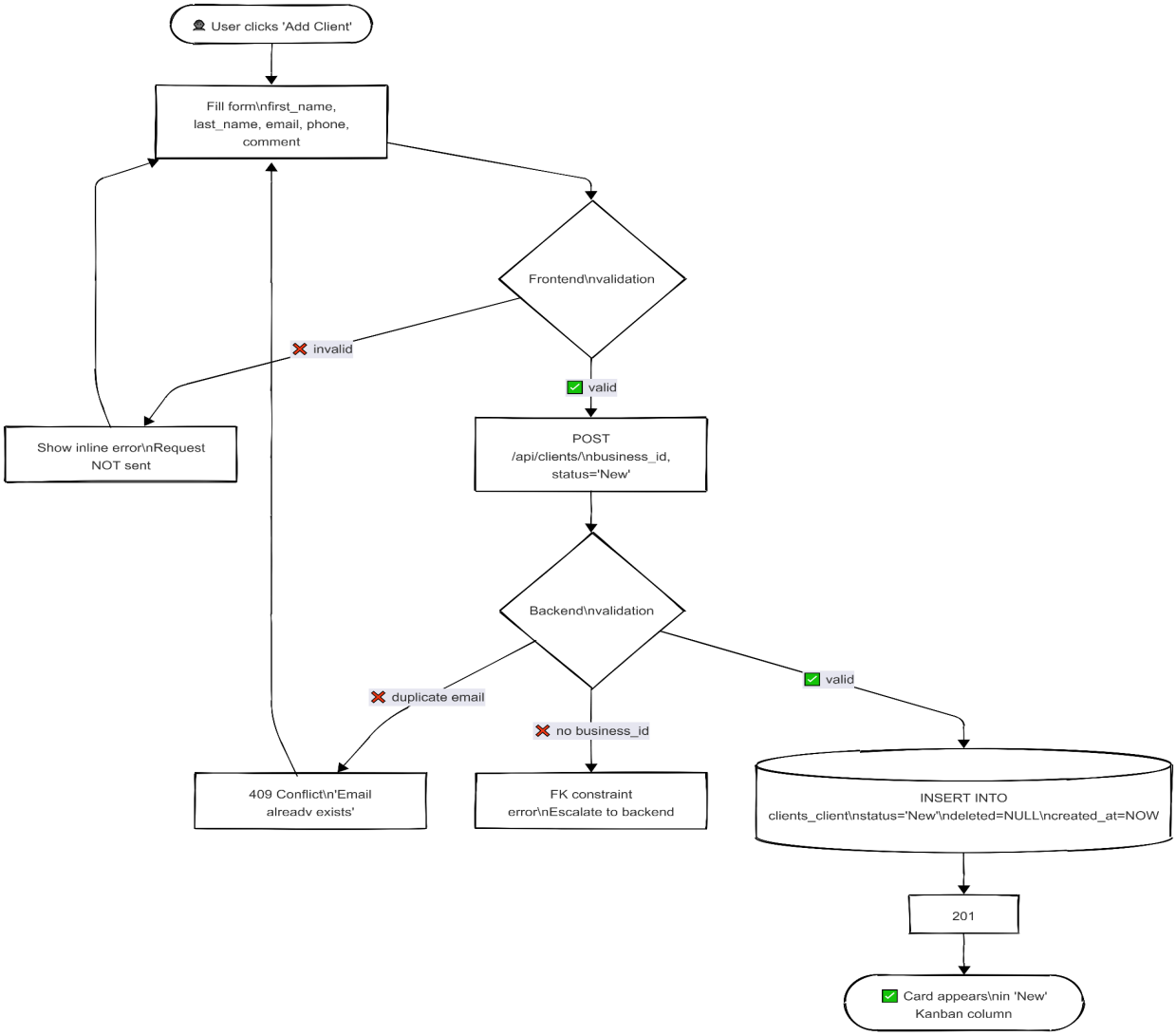
Edge Cases

Scenario	Expected	Actual / Finding
Duplicate email (same business)	409 Conflict	No UNIQUE constraint confirmed at DB level
Empty first/last name	Frontend blocks submit	Should show inline error
Comment > 300 chars	Validation error	⚠️ Query #6 FAILED — id 117 has comment >300 chars
Phone < 7 or > 15 chars	Validation error	⚠️ Query #5 FAILED — id 114 has phone_length = 0
business_id doesn't exist	FK constraint error	⚠️ Query #7 FAILED — 120 orphaned clients exist

Data Integrity Checkpoints

- ✅ status always defaults to 'New' on creation
- ✅ deleted = NULL on creation (not soft-deleted)
- ⚠️ created_at > updated_at anomaly found in 13 records (Query #4 FAILED)
- ⚠️ business_id FK not enforced during seeding/import

Journey 1 — Create New Client



Journey 2: Move Client Through Statuses

Table: clients_client | **DB Operation:** UPDATE

Steps

1. **User Action** —> User drags Kanban card to new column OR selects status from dropdown in Client Details. Allowed values: New → In Progress → Done.
2. **Optimistic UI** —> Card moves instantly. Rollback occurs if API call fails. Target: ≤ 2 seconds.
3. **API Request** — >PATCH /api/clients/:id → Payload: { status: 'In Progress' }.
4. **Backend Validation** —> Client exists (404 if not). Client not soft-deleted (deleted IS NULL). Status value IN ('New', 'In Progress', 'Done'). User belongs to same business_id.
5. **DB Operation** —>UPDATE clients_client SET status='In Progress', updated_at=NOW() WHERE id=:id AND deleted IS NULL AND business_id=:business_id.
6. **Result** → card appears in correct column. Synced in Kanban and table view within ≤ 2 sec.

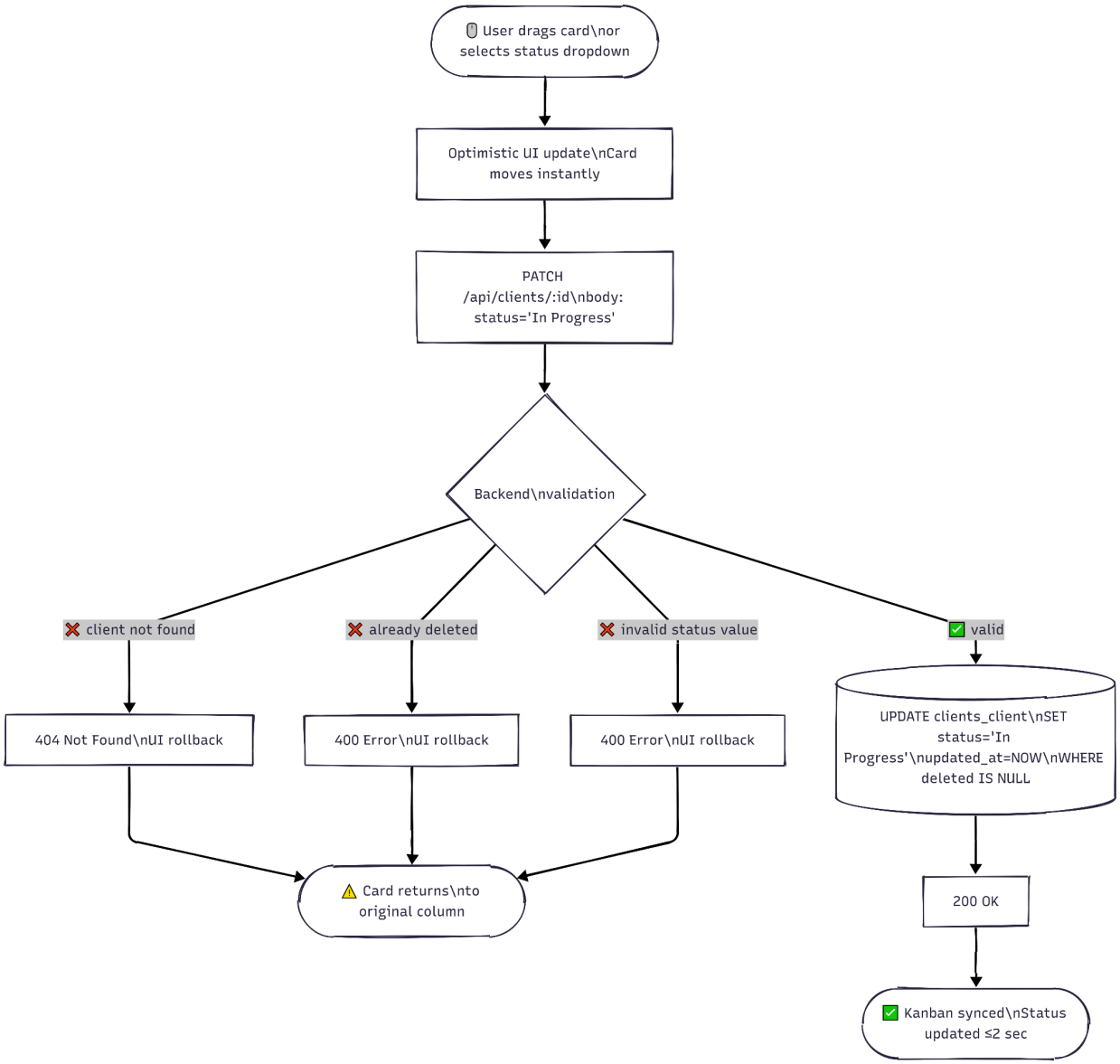
Edge Cases

Scenario	Expected	Actual / Finding
Invalid status value (e.g. 'Pending')	400 error — rejected	✅ Query #3 PASSED — 0 invalid statuses in DB
Moving already-deleted client	404 or 400 error	Blocked by deleted IS NULL check
Network drops mid-update	UI rollback to original status	Depends on optimistic UI implementation
Two users update same client simultaneously	Last-write-wins	Not defined in MVP scope

Data Integrity Checkpoints

- ✅ Only 3 valid status values: New, In Progress, Done
- ✅ updated_at refreshed on every status change
- ⚠️ No audit log / history of status changes stored in DB
- ⚠️ Timestamp anomaly in 13 records may relate to status update bug (Query #4)

Journey 2- Move Client Through Statuses



Journey 3: Add Comment to Client

Table: clients_client | **DB Operation**

Steps

1. **User Action** —> User opens Client Details. Finds Comment field. Types text and clicks Save.
2. **Frontend Validation** —> Comment not empty (min 1 char). Comment ≤ 300 characters. Empty save blocked.
3. **API Request** —> PATCH /api/clients/:id → Payload: { comment: 'text' }. Comment overwrites previous value — NOT appended.
4. **Backend Validation** — >Client exists and not deleted. User belongs to same business. Comment length ≤ 300 chars. Sanitize inputs.
5. **DB Operation** —> UPDATE clients_client SET comment='text', updated_at=NOW() WHERE id=:id AND deleted IS NULL.
6. **Result** → comment displayed in Client Details. Previous comment overwritten.

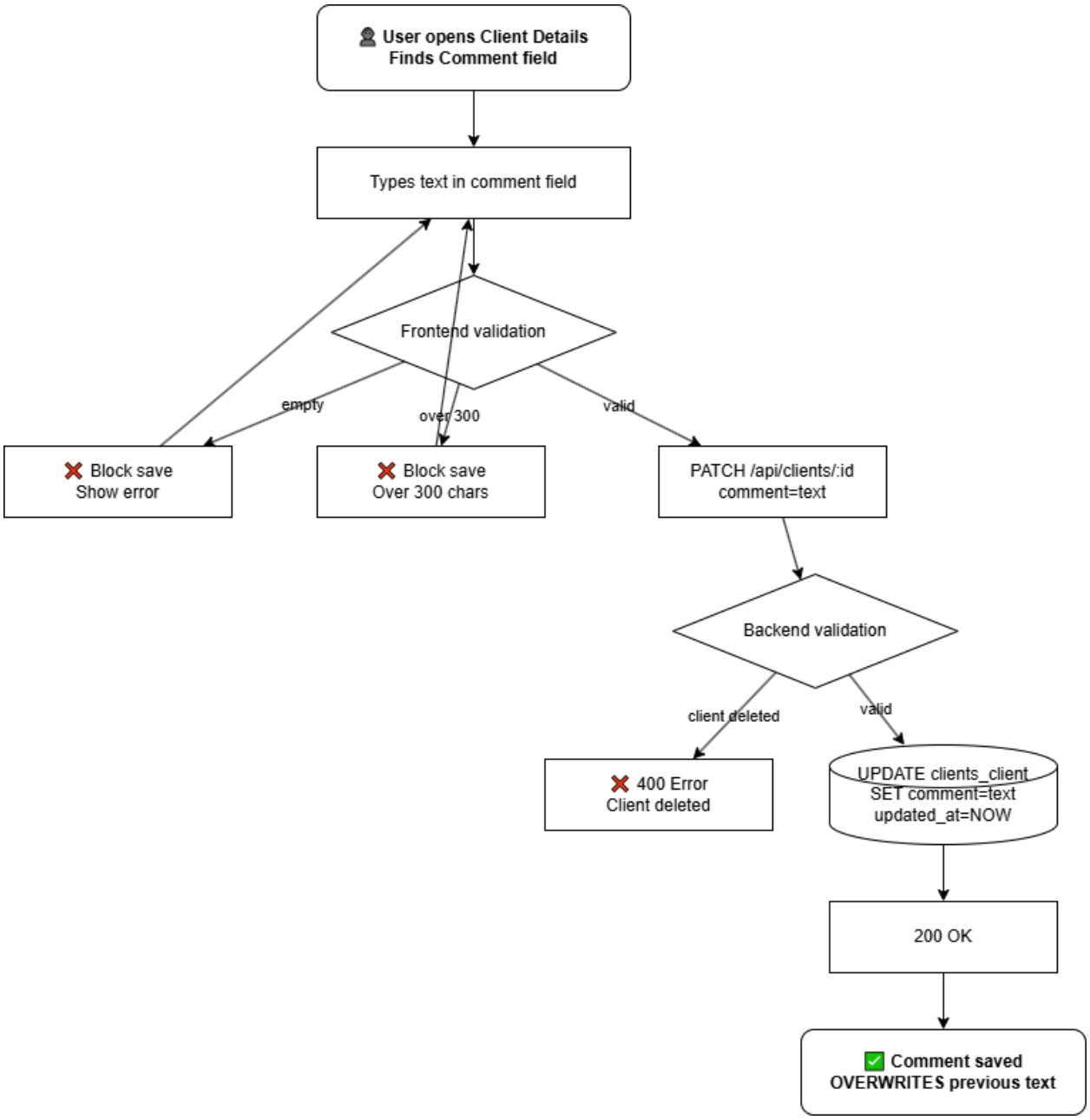
Edge Cases

Scenario	Expected	Actual / Finding
Note > 300 chars	Validation error	⚠️ Query #6 FAILED — id 117 has comment >300 chars
Multiple notes per client	Single comment field	✅ Intentional — simplicity prioritized per benchmarking
Note history tracking	No chronological history	Trade-off: single field reduces complexity in MVP

Data Integrity Checkpoints

- ✅ Notes implemented as comment field — intentional design decision

Journey 3: Add Comment to Client



Journey 4: Soft Delete Client

Table: clients_client | **DB Operation:** UPDATE (soft delete)

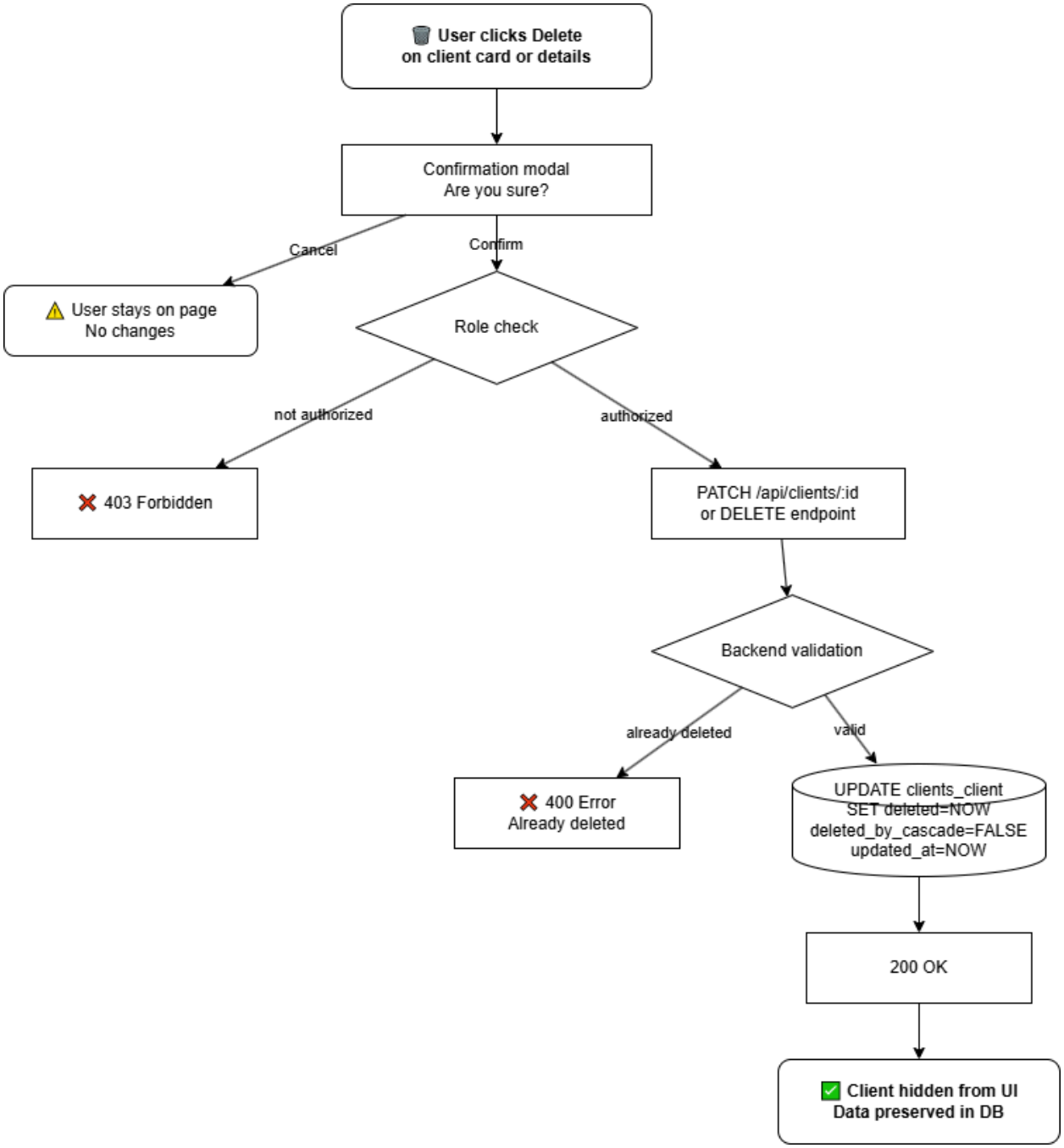
Steps

1. **User Action** —> User clicks 'Delete' on client card or in Client Details. Confirmation modal appears: 'Are you sure?'. User confirms.
2. **Frontend Validation** — >Confirmation modal required. Role check: only authorized user can delete.
3. **API Request** —>PATCH /api/clients/:id or DELETE endpoint → sets deleted field internally.
4. **Backend Validation** — >Client exists (404 if not). Client not already deleted. User authorized for this business_id. Only soft delete allowed — no physical DELETE.
5. **DB Operation** —> UPDATE clients_client SET deleted=NOW(), deleted_by_cascade=FALSE, updated_at=NOW() WHERE id=:id AND deleted IS NULL.
6. **Result** —> Client disappears from Kanban board and Clients table. Data remains in DB (deleted IS NOT NULL). Recoverable by backend if needed.

Edge Cases

Scenario	Expected	Actual / Finding
Delete already-deleted client	400 error or no-op	Blocked by deleted IS NULL check
Unauthorized user tries to delete	403 Forbidden	Role-based restriction per Scope
Cascade delete from business deletion	deleted_by_cascade = TRUE	✅ deleted_by_cascade field exists in schema
Accidental deletion - can undo?	No undo in MVP scope	Data in DB but no UI to restore

Data Integrity Checkpoints

- ✅ Soft delete confirmed: deleted field is datetime (NULL = active, timestamp = deleted)
- ✅ deleted_by_cascade flag distinguishes manual vs cascaded deletion
- ✅ Data preserved in DB after soft delete
- ✅ deleted IS NULL filter used in all active-client queries (confirmed in Queries #1, #2, #3)
- ⚠️ No UI to view or restore deleted clients in MVP scope

Journey 4: Soft Delete Client